

《系统开发工具基础》实验报告



中国海洋大学
OCEAN UNIVERSITY OF CHINA

主题： Shell+git+Latex

学号： 24120011027

姓名： 严程漪

专业： 物流管理

我的 github: [点击这里](#)

2026 年 9 月 1 日

目录

1 实验目的	2
1.1 完成四个课内实验；	2
1.2 了解并学习版本控制 Git；	2
1.3 了解并学习 LaTeX 文本编辑；	2
2 实验内容	2
2.1 学习了 shell 的基础知识。	2
2.2 学习了 Git 的工作原理、基本操作。	2
2.3 学习了用 LaTeX 编写文档，并编写实验报告模板。	2
3 课内实验	2
3.1 第一题：含空格文件名的批量整理	2
3.2 第二题：随机访问日志统计	4
3.3 第三题：制造、解决并解释一次合作冲突	5
3.4 第四题：修复并构建一页技术说明	6
4 课外实例	8
4.1 实例一：ls -l 与前 10 个字符	8
4.2 实例二：home 目录最常见的 5 种扩展名	9
4.3 实例三：stdout/stderr 重定义向	9
4.4 实例四：set -x 调试模式	10
4.5 实例五：awk 过滤与列交换	10
4.6 实例六：带日期的文件备份	11
4.7 实例七：find 与 xargs 结合统计代码行数	11
4.8 实例八：Glob 通用符机制	12
4.9 实例九：克隆库及修改文件	12
4.10 实例十：Git Dotfile 与别名的配置	13
5 解题感悟	14
6 Commit 记录	14

1 实验目的

- 1.1 完成四个课内实验；
- 1.2 了解并学习版本控制 Git；
- 1.3 了解并学习 LaTeX 文本编辑；

2 实验内容

- 2.1 学习了 shell 的基础知识。
- 2.2 学习了 Git 的工作原理、基本操作。
- 2.3 学习了用 LaTeX 编写文档，并编写实验报告模板。

3 课内实验

3.1 第一题：含空格文件名的批量整理

1、创建 input/docs 和 input/tmp；创建 input/docs/notes one.txt（内容为 alpha 和 beta 两行）、input/docs/.secret.txt（内容为 hidden）、input/tmp/empty.txt（空文件）以及 input/run.log（任意两行日志）。

```
ouc@islouc-vm:~$ mkdir -p q01/input/docs q01/input/tmp
ouc@islouc-vm:~$ cd q01
ouc@islouc-vm:~/q01$ printf "alpha\nbeta\n" > input/docs/"notes one.txt"
ouc@islouc-vm:~/q01$ echo "hidden" > input/docs/.secret.txt
ouc@islouc-vm:~/q01$ touch input/tmp/empty.txt
ouc@islouc-vm:~/q01$ printf "log line 1\nlog line 2\n" > input/run.log
```

由于 notes one.txt 文件名中间有空格，所以需要打上双引号。

2、显示 q01 的绝对路径，并以长格式列出 input 下全部项目，包含隐藏文件。

```
ouc@islouc-vm:~/q01$ pwd -P
/home/ouc/q01
ouc@islouc-vm:~/q01$ ls -la input/
total 20
drwxrwxr-x 4 ouc ouc 4096 8月 24 14:52 .
drwxrwxr-x 4 ouc ouc 4096 8月 24 14:53 ..
drwxrwxr-x 2 ouc ouc 4096 8月 24 14:52 docs
-rw-rw-r-- 1 ouc ouc 22 8月 24 15:16 run.log
drwxrwxr-x 2 ouc ouc 4096 8月 24 14:52 tmp
```

pwd 输出绝对路径

-l 是以长格式显示；-a 是显示所有文件，包含隐藏文件

3、把 input 下所有.txt 文件复制到 work/< 学号 >/，保留相对目录结构并正确处理名称中的空格

```
ouc@islouc-vm:~/q01$ mkdir -p work/24120011027
ouc@islouc-vm:~/q01$ find input -type f -name "*.txt" -print0 | xargs -0 -I {} cp --parents {} wor
k/24120011027/
```

首先，先创建一个 work/24120011027 文件

然后，用 find input 从 input 目录开始向下递归搜索；-type f 限定只查找普通文件，忽略目录；-name “*.txt”：按照文件名匹配，查找所有以.txt 结尾的文件；-print0：用空字符作为文件名之间的分隔符；xargs：将管道传过来的内容转化成后续内容的参数；-I：定义一个占位符，后续命令中的都会被替换成具体的某一个文件路径；cp—parents work/24120011027/：实施执行的复制动作，-parents，保留相对目录结构

4、将复制后的目录权限设为 750、普通文件权限设为 640；生成 inventory.txt，列出相对路径和字节数。

```
ouc@islouc-vm:~/q01/work/24120011027$ find input -type d -exec chmod 750 {} +
ouc@islouc-vm:~/q01/work/24120011027$ find input -type f -exec chmod 640 {} +

ouc@islouc-vm:~/q01/work/24120011027$ find input -type f -exec stat -c "%n %s" {} + > inventory.tx
t
```

```
1 input/docs/.secret.txt 7
2 input/docs/notes one.txt 11
3 input/tmp/empty.txt 0
```

-type d：限定只匹配目录；

-exec ... +：批量把所有找到的文件交给后面的命令处理

Stat：Linux 系统自带的用于显示文件或文件系统状态信息的命令

Chmod 750/640 设置权限

3.2 第二题：随机访问日志统计

```
ouc@islouc-vm:~/q01/q02$ cat << 'EOF' > access.csv
> timestamp, user, path, status, latency_ms
> 09:00:01, alice, /api/users, 200, 120
> 09:00:02, bob, /api/orders, 500, 310
> 09:00:03, alice, /api/users, 503, 180
> 09:00:04, carol, /login, 500, 90
> 09:00:05, bob, /api/orders, 502, 260
> 09:00:06, dave, /health, 200, 20
> 09:00:07, alice, /api/orders, 500, 420
> 09:00:08, carol, /api/users, 500, 150
> EOF
```

1、编写 analyze.sh，接受一个 CSV 文件路径；文件不存在时将错误写入 stderr 并返回非零退出码

```
ouc@islouc-vm:~/q01/q02$ cat << 'EOF' > analyze.sh
> if [ ! -f "$1" ]; then
>     echo "Error: File '$1' does not exist." >&2
>     exit 1
> fi
```

2、使用 Shell 管道以及 awk、sort、head 等工具，输出 HTTP 5xx 数量最多的前两个 path；按次数降序，次数相同时按 path 字典序

```
> awk -F ' ' 'NR > 1 && $4 ~ /^[0-9]{2}$/ {count[$3]++} END {for (p in count) print count[p], p}' "$1" | \
> sort -k1,1nr -k2,2 | head -n 2
```

awk 提取与频次相加；

3、输出全部数据行的平均 latency_ms，保留两位小数；表头不得计入。

```
> awk -F ' ' 'NR > 1 {sum += $5; count++} END {if (count > 0) printf "%.2f\n", sum/count}' "$1"
```

4、分别对 access.csv 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

```
ouc@islouc-vm:~/q01/q02$ chmod +x analyze.sh
ouc@islouc-vm:~/q01/q02$ ./analyze.sh access.csv
3 /api/orders
2 /api/users
193.75
ouc@islouc-vm:~/q01/q02$ ./analyze.sh not_exist.csv
Error: File 'not_exist.csv' does not exist.
ouc@islouc-vm:~/q01/q02$ echo $?
1
```

\$? 是一个系统变量，存着上一条命令运行后的退出码

3.3 第三题：制造、解决并解释一次合作冲突

1、初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交。

```
ouc@islouc-vm:~/q03$ git init
Initialized empty Git repository in /home/ouc/q03/.git/
ouc@islouc-vm:~/q03$ echo "mode=normal" > config.txt
ouc@islouc-vm:~/q03$ git add config.txt
ouc@islouc-vm:~/q03$ git commit -m "Initial commit"
[master (root-commit) db1196e] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 config.txt
```

2、创建 feature-a，把内容改成 mode=safe 并提交；从初始 main 创建 feature-b，把内容改成 mode=fast 并提交。

```
ouc@islouc-vm:~/q03$ git checkout -b feature-a
Switched to a new branch 'feature-a'
ouc@islouc-vm:~/q03$ echo "mode=safe" > config.txt
ouc@islouc-vm:~/q03$ git commit -am "Set mode=safe"
[feature-a 7bfc6c4] Set mode=safe
1 file changed, 1 insertion(+), 1 deletion(-)
ouc@islouc-vm:~/q03$ git branch
* feature-a
  master
ouc@islouc-vm:~/q03$ git checkout master
Switched to branch 'master'
ouc@islouc-vm:~/q03$ git checkout -b feature-b
Switched to a new branch 'feature-b'
ouc@islouc-vm:~/q03$ echo "mode=fast" > config.txt
ouc@islouc-vm:~/q03$ git commit -am "Set mode=fast"
[feature-b 3d43753] Set mode=fast
1 file changed, 1 insertion(+), 1 deletion(-)
```

3、回到 main，先合并 feature-a，再合并 feature-b，解决冲突时保留 mode=safe，并另加一行 note=reviewed
合并 feature-a

```
ouc@islouc-vm:~/q03$ git checkout main || git checkout master
error: pathspec 'main' did not match any file(s) known to git
Switched to branch 'master'
ouc@islouc-vm:~/q03$ git merge feature-a
Updating db1196e..7bfc6c4
Fast-forward
 config.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

合并 feature-b

```
ouc@islouc-vm:~/q03$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
```

解决冲突时保留 mode=safe，并另加一行 note=reviewed

```
ouc@islouc-vm:~/q03$ printf "mode=safe\nnote=reviewed\n" > config.txt
ouc@islouc-vm:~/q03$ git add config.txt
ouc@islouc-vm:~/q03$ git commit -m "Merge feature-b and resolve conflict"
[master b8e6649] Merge feature-b and resolve conflict
```

4、确认工作区干净，运行 `git log -all -graph -decorate -oneline` 查看历史

```
ouc@islouc-vm:~/q03$ git log --all --graph --decorate --oneline
* b8e6649 (HEAD -> master) Merge feature-b and resolve conflict
| \
| * 3d43753 (feature-b) Set mode=fast
* | 7bfc6c4 (feature-a) Set mode=safe
|/
* db1196e Initial commit
```

3.4 第四题：修复并构建一页技术说明

1、加入一个带编号和 label 的公式，并在正文中使用 ref 或 eqref

```
> As shown in Equation~\eqref{eq:einstein}, the energy-mass equivalence is fundamental.
> \begin{equation}
> \label{eq:einstein}
> E = mc^2
> \end{equation}
```

2、加入一个 2 行 2 列表格，设置 caption 和 label，并在正文引用该表。

```

> The experimental data is summarized in Table~\ref{tab:data}.
> \begin{table}[htbp]
> \centering
> \caption{Sample Data Table}
> \label{tab:data}
> \begin{tabular}{|c|c|}
> \hline
> Item & Value \\
> \hline
> Alpha & 100 \\
> \hline
> \end{tabular}
> \end{table}

```

3、使用 `latexmk -pdf -halt-on-error` 或 `tectonic` 生成 `report.pdf`。

```

=== TeX engine is 'pdfTeX'
Latexmk: Log file says output to 'report.pdf'
Latexmk: All targets () are up-to-date

```

4、确定 PDF 能打开，且交叉引用不显示 “??”

Tool Report

yanchengyi-24120011027

August 24, 2026

1 Result

As shown in Equation (1), the energy-mass equivalence is fundamental.

$$E = mc^2 \tag{1}$$

The experimental data is summarized in Table 1.

Table 1: Sample Data Table

Item	Value
Alpha	100

4 课外实例

4.1 实例一：ls -l 与前 10 个字符

1、-l 是 long format (长格式)：每个文件一行，依次显示权限、硬链接数、属主、属组、大小、修改时间、文件名。

```
ouc@islouc-vm:~$ ls -l /
total 2097232
lrwxrwxrwx   1 root root          7 9月 25  2023 bin -> usr/bin
drwxr-xr-x   4 root root     4096 5月 12 08:15 boot
drwxrwxr-x   2 root root     4096 9月 25  2023 cdrom
drwxr-xr-x  18 root root    4180 8月 31 01:09 dev
drwxr-xr-x 146 root root   12288 8月 31 13:42 etc
drwxr-xr-x   3 root root     4096 9月 25  2023 home
lrwxrwxrwx   1 root root          7 9月 25  2023 lib -> usr/lib
lrwxrwxrwx   1 root root          9 9月 25  2023 lib32 -> usr/lib32
lrwxrwxrwx   1 root root          9 9月 25  2023 lib64 -> usr/lib64
lrwxrwxrwx   1 root root         10 9月 25  2023 libx32 -> usr/libx32
drwx-----  2 root root   16384 9月 25  2023 lost+found
drwxr-xr-x   2 root root     4096 2月 23  2022 media
drwxr-xr-x   2 root root     4096 2月 23  2022 mnt
drwxr-xr-x   4 root root     4096 10月 18  2023 opt
dr-xr-xr-x 386 root root         0 8月 24 14:42 proc
drwx-----  7 root root     4096 10月 18  2023 root
drwxr-xr-x  39 root root    1100 9月  1 08:35 run
lrwxrwxrwx   1 root root          8 9月 25  2023/sbin -> usr/sbin
drwxr-xr-x  13 root root     4096 8月 31 12:02 snap
drwxr-xr-x   2 root root     4096 2月 23  2022 srv
-rw-----  1 root root 2147483648 9月 25  2023 swapfile
dr-xr-xr-x  13 root root         0 8月 24 14:42 sys
drwxrwxrwt  20 root root     4096 9月  1 18:00 tmp
drwxr-xr-x  14 root root     4096 2月 23  2022 usr
drwxr-xr-x  15 root root     4096 9月 25  2023 var
```

2、前 10 个字符的含义

位置	含义
第 1 位	文件类型：- 普通文件，d 目录，l 符号链接，c 字符设备，b 块设备，p 管道，s 套接字
第 2-4 位	属主 (owner) 的 r 读、w 写、x 执行权限
第 5-7 位	属组 (group) 的 rwx
第 8-10 位	其他人 (others) 的 rwx

4.2 实例二：home 目录最常见的 5 种扩展名

```
ouc@islouc-vm:~$ find ~ -type f 2>/dev/null | sed 's|./|'| | grep '\.' | sed 's|./|.|' | sort | uniq -c | sort -rn | head -5
15418 pyi
15054 asm
5949 py
4341 md
3728 pyc
```

4.3 实例三：stdout/stderr 重定向

1、分别重定向 stdout 和 stderr 到不同文件：

```
ls /nonexistent /tmp > out.txt 2> err.txt
```

2、合并重定向 stdout 和 stderr 到同一个文件：

```
ls /nonexistent /tmp > all.txt 2>&1
```

```
ouc@islouc-vm:~$ cat out.txt
/tmp:
config-err-oqLXyt
snap-private-tmp
ssh-oZg2NOP5jAYF
systemd-private-0fc988bd939344aaab83dae6419e77a8-colord.service-i6HQNi
systemd-private-0fc988bd939344aaab83dae6419e77a8-fprintd.service-r7DkHf
systemd-private-0fc988bd939344aaab83dae6419e77a8-ModemManager.service-OayDhi
systemd-private-0fc988bd939344aaab83dae6419e77a8-switcheroo-control.service-olz2Ph
systemd-private-0fc988bd939344aaab83dae6419e77a8-systemd-logind.service-uKgW1f
systemd-private-0fc988bd939344aaab83dae6419e77a8-systemd-resolved.service-jYY21g
systemd-private-0fc988bd939344aaab83dae6419e77a8-systemd-timesyncd.service-9Abxjh
systemd-private-0fc988bd939344aaab83dae6419e77a8-upower.service-YLGyrj
tracker-extract-files.1000
tracker-extract-files.125
VMwareDnD
vmware-root_775-4248221734
```

```
ouc@islouc-vm:~$ cat err.txt
ls: cannot access '/nonexistent': No such file or directory
```

```
ouc@islouc-vm:~$ cat all.txt
ls: cannot access '/nonexistent': No such file or directory
/tmp:
config-err-oqLXyt
snap-private-tmp
ssh-oZg2NOP5jAYF
systemd-private-0fc988bd939344aaab83dae6419e77a8-colord.service-i6HQNi
systemd-private-0fc988bd939344aaab83dae6419e77a8-fprintd.service-r7DkHf
systemd-private-0fc988bd939344aaab83dae6419e77a8-ModemManager.service-OayDhi
systemd-private-0fc988bd939344aaab83dae6419e77a8-switcheroo-control.service-olz2Ph
systemd-private-0fc988bd939344aaab83dae6419e77a8-systemd-logind.service-uKgW1f
systemd-private-0fc988bd939344aaab83dae6419e77a8-systemd-resolved.service-jYY21g
systemd-private-0fc988bd939344aaab83dae6419e77a8-systemd-timesyncd.service-9Abxjh
systemd-private-0fc988bd939344aaab83dae6419e77a8-upower.service-YLGyrj
tracker-extract-files.1000
tracker-extract-files.125
VMwareDnD
vmware-root_775-4248221734
```

4.4 实例四：set -x 调试模式

1、作用：启用 xtrace 模式。Shell 会在执行每条命令前，先将变量展开、引号处理后的实际执行命令打印到标准错误 stderr。

2、测试脚本与运行

```
ouc@islouc-vm:~$ cat > trace.sh << 'EOF'
> set -x
> name="world"
> echo "Hello, $name"
> for i in 1 2; do
>   echo "loop $i"
> done
> set +x
> echo "追踪已关闭"
> EOF
ouc@islouc-vm:~$ chmod +x trace.sh
```

```
ouc@islouc-vm:~$ ./trace.sh
++ name=world
++ echo 'Hello, world'
Hello, world
++ for i in 1 2
++ echo 'loop 1'
loop 1
++ for i in 1 2
++ echo 'loop 2'
loop 2
++ set +x
追踪已关闭
```

3、观察结果：在命令真正输出结果前，均会以 + 开头的被解析后的命令行，清晰展示变量 \$ 被替换成 world 的过程。最后通过 set +x 关闭跟踪。

4.5 实例五：awk 过滤与列交换

1、测试命令：

```
printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2 > 100 {t=$1; $1=$3; $3=t; p
```

```
ouc@islouc-vm:~$ printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '{t=$1; $1=$3; $3=t; p
$3=t; print}'
y 150 b
z 200 c
```

4.6 实例六：带日期的文件备份

1、基础命令

```
touch notes.txt
```

```
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

```
ouc@islouc-vm:~$ touch notes.txt
ouc@islouc-vm:~$ cp notes.txt "notes_$(date +%Y-%m-%d).txt"
ouc@islouc-vm:~$ ls
all.txt          my_cp.c          Public           test_link
BurpSuiteCommunity  my_cp.c.save    q01             test_no
copy_of_my_cp.c   my_cp.c:wq      q02             test_nx_default
Desktop          my_hole         q03             test_nx_disabled
Documents        my_hole.c       q04             test_nx_enabled
Downloads        my_link.c       q06             test_pie_1
err.txt          my_ls           q07             test_pie_2
ghidra_10.3.3_PUBLIC my_ls.c        q08             test_pie_default
hole.cat         my_pipe         snap            test_pro
hole.cp          my_type         source.txt      test_relro_full
hole.file        my_type.c       target.txt      test_relro_no
hybrid_crypto.py my_unlink       Templates       test_relro_partial
idafree-8.3       my_unlink.c     test_all        test.txt
Music            notes_2026-09-01.txt test.c          trace.sh
my_chmod         notes.txt       test_fortify_0  Videos
my_chmod.c       os_experiment   test_fortify_1
my_commands.txt  out.txt        test_fortify_2
my_cp            Pictures       test_fortify_3
```

2、命令替换原理

`$(date +%Y-%m-%d)` 会优先执行并获取格式化“年-月-日”的当前日期，再拼接到新文件名中

4.7 实例七：find 与 xargs 结合统计代码行数

1、可以安全处理空格的命令如下：

```
find . -type f -name '*.sh' -print0 | xargs -0 wc -l
```

2、实现机制：`-print0` 指定 `find` 使用 NUL 字符 (`\0`) 作为分隔符；`-0` 指定 `xargs` 按 NUL 字符解析参数。由于 NUL 是 Linux 文件系统中唯一禁止出现的字符，因此能解决文件名带空格或换行带来的解析错乱问题。

4.8 实例八：Glob 通用符机制

1、Glob 含义：Glob 是 Shell 内部用于匹配文件名模式的通配符机制。在命令执行前，Shell 会自动将其展开为符合条件的文件列表再传递给程序。

2、文件的创建与现象观察

```
ouc@islouc-vm:~$ mkdir ~/globtest && cd ~/globtest
ouc@islouc-vm:~/globtest$ touch a.txt b.txt c.txt file1.txt file2.txt file10.txt notes.md
ouc@islouc-vm:~/globtest$ ls *.txt
a.txt b.txt c.txt file10.txt file1.txt file2.txt
ouc@islouc-vm:~/globtest$ ls file?.txt
file1.txt file2.txt
ouc@islouc-vm:~/globtest$ ls {a,b,c}.txt
a.txt b.txt c.txt
ouc@islouc-vm:~/globtest$ ls file[12].txt
file1.txt file2.txt
ouc@islouc-vm:~/globtest$ ls {a,d}.txt
ls: cannot access 'd.txt': No such file or directory
a.txt
ouc@islouc-vm:~/globtest$ echo *.txt
a.txt b.txt c.txt file10.txt file1.txt file2.txt
```

4.9 实例九：克隆库及修改文件

1、原本是需要从 GitHub 中克隆，但由于虚拟机连不上 GitHub，所以使用纯本地新建仓库，命令流程一致

```
ouc@islouc-vm:~/globtest$ curl -I https://github.com
curl: (28) Failed to connect to github.com port 443: Connection timed out
```

2、创建并进入测试仓库

```
mkdir ~/git-test && cd ~/git-test
git init
echo "Hello World" > README
git add README
git commit -m "Initial commit"
```

3、修改文件并查看状态

```
echo "my change" >> README
git status
```

```
ouc@islouc-vm:~/git-test$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README

no changes added to commit (use "git add" and/or "git commit -a")
```

4、修改文件后运行 git stash 会发生什么？

Git 会将当前工作区中未添加的修改保存到一个特殊的暂存栈中，同时将工作区重置恢复到最近一次提交的状态，使工作区变回干净状态。

```
ouc@islouc-vm:~/git-test$ git stash
Saved working directory and index state WIP on master: ef43b5b Initial commit
```

5、运行 git log -all --oneline 会看到什么？为什么？

看到的现象：日志中新增了指向 refs/stash 的条目

底层原理解释：git stash 在 Git 底层上实际是由 Git 自动创建的特殊提交对象。其中一个提交保存暂存区状态，另一个提交保存工作区状态，并由 refs/stash 引用指针指向它们。由于使用了 -all 参数，git log 会遍历包括 refs/stash 在内的所有引用，因此能够直接看到这个悬挂在后台的暂存提交树。

```
ouc@islouc-vm:~/git-test$ git log --all --oneline
9c9f708 (refs/stash) WIP on master: ef43b5b Initial commit
3bc2015 index on master: ef43b5b Initial commit
ef43b5b (HEAD -> master) Initial commit
```

6、运行 git stash pop 会发生什么？

它会把位于暂存栈顶部的修改重新恢复并应用到当前工作区中，同时将该 refs/stash@0 节点从 stash 栈中彻底弹出并删除。

```
ouc@islouc-vm:~/git-test$ git stash pop
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (9c9f7080f53ca5a039a05cc3898a9cdf90d71528)
```

7、在什么情况下 git stash 会非常有用？

- 1、临时切换分支处理紧急 Bug：当前分支功能只开发了一半，无法生成规范的 commit，但需要立即切去其他分支修复紧急线上 Bug 时。
- 2、解决 git pull 冲突：拉取远程最新代码时被提示“本地存在未提交的修改而拒绝 pull”，可先 stash 暂存，拉取成功后再 pop 还原。
- 3、修复误操作的分支：发现自己不小心把原本属于 feature 分支的代码写在了 main 分支上，可先 stash，切换回 feature 分支后再 pop 出来。

4.10 实例十：Git Dotfile 与别名的配置

1、使用 Git 命令行配置

```
git config --global alias.graph "log --all --graph --decorate --oneline"
```

2、运行验证与输出结果

```
ouc@islouc-vm:~/git-test$ git graph
* ef43b5b (HEAD -> master) Initial commit
ouc@islouc-vm:~/git-test$
```

5 解题感悟

本次 Linux 与 Git 基础实验通过管道命令组合、标准流重定向与 Shell 通配符展开，深化了我对 Linux 模块化设计和工作流控制的理解；同时借助 git stash 与别名配置，我直观窥探到了 Git 底层 Commit 对象的运作机制。实践不仅提升了我的命令行实操能力，也培养了通过日志和原理排查问题的思维模式。

6 Commit 记录

 Yan-cheng-yi	Add files via upload	88c2ba4 · 2 minutes ago	
 paper	Add files via upload	5 hours ago	
 第一次课后实验结果图	Add files via upload	45 minutes ago	
 README.md	Initial commit	8 hours ago	
 第一次课后作业命令记...	Add files via upload	2 minutes ago	
 系统开发工具基础第一...	Add files via upload	3 hours ago	
 系统开发工具基础第一...	Add files via upload	8 hours ago	
 系统开发模板.zip	Add files via upload	4 minutes ago	